

SAM-V1: Generative AI Architecture (Interview Guide)

This document outlines the core concepts of the current SAM-V1 architecture. This is a Hybrid Generative Model: it uses hardcoded tools for specific commands, and a Generative LLM fallback for conversational queries.

1. Speech-to-Text (STT) & Text-to-Speech (TTS)

Used Python's SpeechRecognition library to capture microphone input and route it against Google's API for transcription. Used Pyttsx3 for offline, zero-latency voice generation (TTS), ensuring immediate audible feedback without network overhead.

2. Passive vs Active Listening State

Implemented a dual-state listening loop to save resources. A passive loop listens globally exclusively for a Wake-Word ('sam'). Once triggered, it elevates to an Active listening state with a strict phrase time limit to capture the actual command.

3. Centralized Routing (Switchboard Pattern)

Moved away from a monolithic, high cyclomatic-complexity file (massive if/elif chains in main) to a modular routing table. The command string is evaluated by 'router.py', cleanly dispatching execution to isolated modules (system.py, web.py).

4. Generative Fallback Mechanism

Integrated the Gemini API. If the routing table determines the user's command is not a local system instruction, the query is passed to the LLM. This provides a flawless illusion of intelligence, bridging rigid scripts with fluid conversation.